Reviewer Pack — Real Root Count Exponential in AC⁰ PARITY Circuits

Entry a1e4626e2b6a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 07:15:38 UTC

Real Root Count Exponential in AC⁰ PARITY Circuits

Entry ID: `a1e4626e2b6a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 07:15:38 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): AC⁰ Circuit Size for PARITY

Statement:

> For any AC⁰ circuit C computing PARITY on n variables, the number of real roots of the polynomial P_C derived from C is Θ(2^{n/2})

Rationale (proposer's reasoning):

> The parity function's Fourier transform has specific properties, and the number of real roots of P_C could reflect the circuit's structure, potentially exposing lower bounds via real algebraic geometry

Taxonomy category: `AC0_PARITY` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f0be56034de1a185`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def sturm_sequence(poly, deriv):
        seq = [poly]
        while True:
            if not deriv:
                break
            seq.append([-deriv[0]/seq[-1][0]] + [b/a for a, b in zip(seq[-2], deriv[1:])])
            deriv = [b - (a * seq[-1][0]) for a, b in zip(seq[-1], deriv)]
        return seq

    def sturm_count(poly):
        seq = sturm_sequence(poly, poly_derivative(poly))
        sign_changes = 0
        for interval in range(-1000, 1001, 2):
            signs = [int(math.copysign(1, p.subs(x, interval))) for p in seq]
            sign_changes += sum(signs[i] != signs[i+1] for i in range(len(signs)-1))
        return sign_changes

    def poly_derivative(poly):
        return [(i * coeff) for i, coeff in enumerate(poly[1:], start=1)]

    def cnf_to_poly(cnf):
        n = len(cnf)
        poly = [Fraction(1)]
        for clause in cnf:
            term = Fraction(1)
            for literal in clause:
                if literal > 0:
                    term *= (1 - x**literal)
                else:
                    term *= (1 + x**(-literal))
            poly.append(term)
        return poly

    def is_ac0_circuit(cnf):
        # Placeholder function to check if the CNF is an AC⁰ circuit
        # This is a dummy implementation and should be replaced with actual logic
        return False
```

```
    n = random.choice([5, 10, 15, 20, 30, 40])
    cnf = [[random.randint(1, n) for _ in range(random.randint(1, n))] for _ in range(n)]

    poly = cnf_to_poly(cnf)
    root_count = sturm_count(poly)

    if is_ac0_circuit(cnf):
        expected_root_count = 2**(n//2)
        conjecture_holds = abs(root_count - expected_root_count) <= 1
        counterexample = "" if conjecture_holds else "AC⁰ circuit does not match expected root cou
    else:
        expected_root_count = 0
        conjecture_holds = root_count == expected_root_count
        counterexample = "" if conjecture_holds else "Non-AC⁰ circuit has unexpected root count"

    return {
        "metric_name": "Root Count",
        "metric_value": root_count,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"AC⁰ circuit does not match expected root count\
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5fd1301b.py", line 88, in <module>
    result = run_trial(seed)
```

```
            ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5fd1301b.py", line 62, in run_trial
    poly = cnf_to_poly(cnf)
           ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5fd1301b.py", line 48, in cnf_to_poly
    term *= (1 - x**literal)
                 ^
NameError: name 'x' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to undefined variable 'x' in polynomial construction, preventing evaluation of root counts. | next: Fix the polynomial construction code to define variables properly and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109830
2	propose	ollama_remote	qwen3:8b	0	0	109173
3	preregistration	ollama_remote	qwen3:8b	0	0	24109
4	novelty	ollama_remote	qwen3:8b	0	0	21508
5	novelty	ollama_remote	qwen3:8b	0	0	14128
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15744
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10444
8	judge	ollama_remote	qwen3:8b	0	0	18335

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 323271 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/a1e4626e2b6a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a1e4626e2b6a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a1e4626e2b6a.tar.gz` (if generated)